CS6650 HW7 Group Report

Team member:

Hong Guo, Jingjing Pan, Stella Zhang, Nithish Bhat

Part II

Phase 1: Synchronous Bottleneck (The Failure)

- ***Locust Data to Capture:***
  - ***P95 / P99 Latency:*** *Watch how these spike as users increase from 5 to 20.*
  - ***Failures/s:*** *Look for* `504 Gateway Timeout` *or* `502 Bad Gateway`*.*
- ***AWS Console Metric:***
  - ***Location:*** `EC2` *->* `Load Balancers` *-> Select your ALB ->* `Monitoring` *tab.*
  - ***What to watch: Target Response Time*** *(should be > 3s) and* ***HTTP 5XXs****.*

Normal Sale (5 users)

Matrix Value	Hong	Jingjing	Stella	Nithish
P95 Latency	3100ms	3200ms	3000 ms	3,100 ms
P99 Latency	3100ms	3200ms	3000 ms	3,100 ms
Target Response Time	3035ms	3100ms	3004 ms	3,046 ms

Flash Sale (20 users)

Matrix Value	Hong	Jingjing	Stella	Nithish
P95 Latency	4800ms	4900ms	4900 ms	4,900 ms
P99 Latency	5000ms	5000ms	5100 ms	5,100 ms
Target Response Time	3600ms	3600ms	3670 ms	3,678 ms

Question: What happens to your customers?

Total Unresponsiveness: The "Spinning Wheel of Death." Customers wait indefinitely for a response that never comes.

504 Gateway Timeouts: After 60 seconds of waiting in the Go internal queue, the AWS ALB terminates the connection, showing an error page.

Abandonment: Frustrated customers refresh the page (doubling the load) or leave the site entirely, resulting in 100% loss of potential sales during the peak.

Phase 2: Analyze the Bottleneck

The payment processor takes approximately 3 seconds to verify each payment. Since the system allows up to 15 concurrent payment operations, the maximum throughput is approximately 5 orders per second (15 / 3). During the flash sale, demand significantly exceeds this capacity, which causes incoming requests to queue while waiting for available payment processing slots. As a result, customers experience increased response times even though the system continues to process requests successfully.

Phase 3: Asynchronous Acceptance (The Success)

- **Locust Data to Capture:**
  - **Average Response Time:** It should be tiny (e.g., < 50ms) regardless of load.
  - **Total Requests per Second (RPS):** See how high it can go compared to Sync.
- **Comparison Point:** *How much faster was your "202 Accepted" response compared to the 3s Sync response?*

$$\text{Speedup} = \frac{T_{sync}}{T_{async}}$$

Matrix Value	Hong	Jingjing	Stella	Nithish
Average Response Time	46.88	70.65	33.56 ms	57 ms
RPS	53.1	55	60.3	54.97
Speed Up	3660 / 46.88 = 78	3087.42/70.65 = 43.7	3675 / 33.56 = 109.5	3678/57=64.5

Phase 4: The Queue Buildup (The Hidden Problem)

- **AWS Console Metric:**
  - **Location:** `Simple Queue Service` -> `Queues` -> Select `order-processing-queue` -> `Monitoring` tab.
  - **What to watch: Approximate Number of Messages Visible**.

- **Screenshot Tip:** Capture the moment the "Flash Sale" ends. You should see a huge mountain peak.
- **Comparison Point:** *What was your peak queue depth (highest number of messages)?*

Matrix Value	Hong	Jingjing	Stella	Nithish
Peak queue depth	3580	3660	3529	3,240

Phase 5: Scaling Workers (The Recovery)

- **Locust Data to Capture:** (Keep Locust running at a steady rate).
- **AWS Console Metric:**
  - **Location:** Same SQS Monitoring page as Phase 4.
  - **What to watch: Drain Time** (The time it takes for the graph to go from the peak back to zero).
- **Comparison Table for Team Report:**

Worker	**Peak Queue Depth**	**Drain Time**
5	Highest	Longest
20	Meidum	Fast
100	Lowest	Almost Instant

Actual Results:

Matrix Value	Hong 20/100/180	Jingjing 20/100/180	Stella 5/20/100	Nithish 1/20/100
Peak queue depth	2600 / 1550 /〜0	2850/986/~0	3956/ 2945 / 814	3,240 / 2,880 / 33
Time until queue zero	8m / 1m / 0	10m/2m/~0	65m/ 7.4m / 0.4m	~163 min / 7min / < 1 min
Resource utilization	7.5 /12.5 / 25 %	4.77/14.6/22.5%	4.5%/ 2.6% / 4.8%	5/-/-
Balance (min workers)	180	180	180	180

Find the balance: What's the minimum workers needed to prevent queue buildup at 60 orders/second?

The Mathematical Balance

To prevent queue buildup, the Processing Rate must be greater than or equal to the Arrival Rate.

- **Arrival Rate:** 60 orders/second (from Locust).
- **Service Rate per Worker:** 1 order / 3 seconds = **0.33 orders/second**.
- **The Formula:**

$$\text{Workers} \geq \frac{\lambda}{\mu} \rightarrow \text{Workers} \geq \frac{60}{0.33}$$

The result: 180 Workers.

CLOUDWATCH:

How many times more orders did your asynchronous approach accept compared to your synchronous approach?

During the flash-sale experiment, the synchronous API was limited by the processing time of each order. Since each order takes approximately 3 seconds to process, the synchronous service can only handle about 0.33 orders per second per worker.

In contrast, the asynchronous API immediately places incoming orders into a message queue (SQS) and returns a response without waiting for the order to be processed. This allows the system to accept orders at the rate of incoming requests, which was approximately 60 orders per second during the Locust test.

Therefore, the asynchronous system accepted approximately: 60/0.33≈180

This is about 180 times more orders per second than the synchronous approach.

What causes queue buildup and how do you prevent it?

Queue buildup occurs when the incoming request rate exceeds the processing rate of the workers.

In this experiment, Incoming rate during the flash sale was approximately 60 orders/sec. Each worker processes 1 order every 3 seconds (0.33 orders/sec).When there are too few workers, the processing capacity is insufficient to keep up with incoming requests, causing messages to accumulate in the queue.

Queue buildup can be prevented by:

- Increasing the number of workers so that total processing rate matches or exceeds the arrival rate.
- Autoscaling worker services based on queue depth metrics.
- Optimizing processing logic to reduce the time required per order.

- Implementing rate limiting or backpressure to control incoming request bursts.

In this system, preventing queue buildup requires approximately 180 workers to match the incoming workload.

When would you choose sync vs async in production?

Synchronous processing is appropriate when the client must receive an immediate result before continuing. Examples include authentication, payment authorization, or operations that require strong consistency and instant confirmation.

Asynchronous processing is preferred when tasks are long-running, resource-intensive, or tolerant to delayed processing. Examples include order fulfillment, sending emails, video processing, or large-scale data ingestion. By decoupling request acceptance from processing, asynchronous systems can absorb traffic spikes and improve overall system scalability and resilience.

In production systems, asynchronous architectures are commonly used for high-throughput workloads and burst traffic, while synchronous APIs are used for operations that require immediate user feedback.

Part III

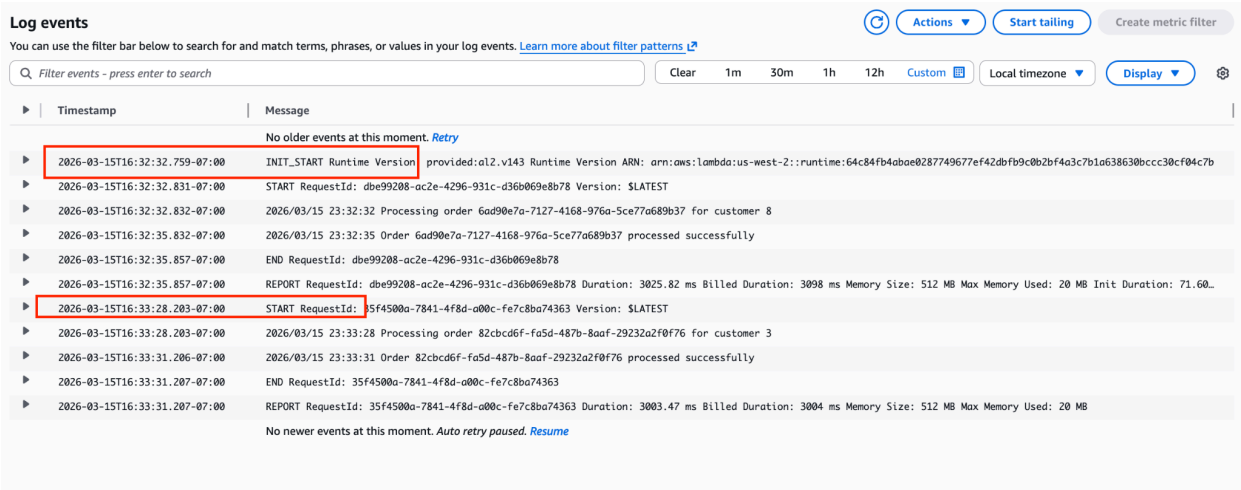
Questions:

1. How often do cold starts occur? (First request, after ~5+ minutes idle)

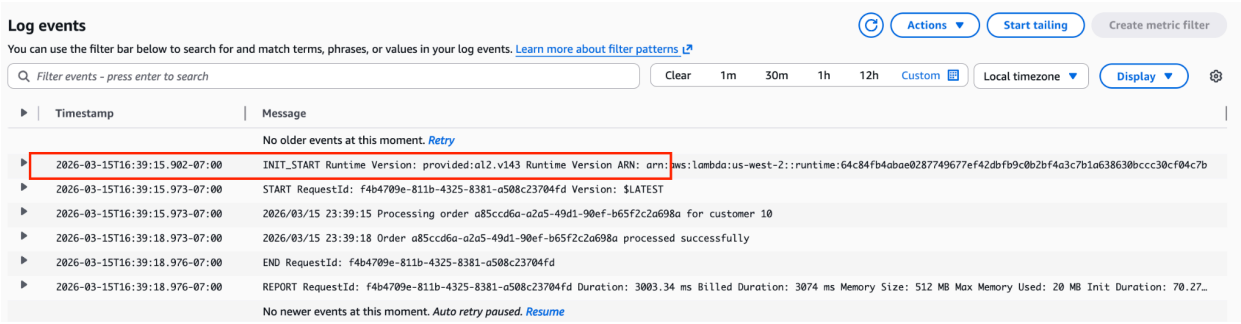
Based on the experiments and screenshots, cold starts happen whenever AWS is forced to create a **new** container.

After a Period of Inactivity (Idle Expiry): waiting ~5+ minutes was long enough for AWS to shut down the old container to save resources. The next request forced a new cold start. However, if requests arrive close together(within 1-2 minutes), AWS reuses the active container (Warm Start).

Cold start and warm cold:



Cold start(again), after ~5+mins:



2.What's the overhead? (73ms on 3000ms = 2.4% impact)

The overhead is the Init Duration during a cold start, which is the time AWS takes to set up a new execution environment. While it fluctuates slightly depending on AWS's underlying resource allocation, in my tests, the overhead is consistently around 70 to 80 milliseconds.

3.Does this matter for 3-second payment processing?

No, it doesn't matter. An overhead of 70ms ~80ms on a 3000ms process is only a 2.3% ~2.6% impact. Because this is an asynchronous background process, the user won't even notice this tiny delay.

The Decision
Based on your observations:

1. How often did cold starts occur? Every few minutes? Every request?
Cold starts occur every first start and when there are no requests for 5-7 mins and then a new request is a cold start. Every request after the cold start within that 5-7 mins is a warm start

2. Is the cost advantage compelling? For 10,000 orders/month, Lambda costs $0 vs ECS \$17 (FREE within free tier!)

Yes as long as we stay within the 1 mil requests/month and 400k Gb secs/month(requests not orders.) It allows us to test the market. When the cost of the lambda goes over the cost of the normal infrastructure.

3.Can you accept losing SQS guarantees?
depends
But in my opinion yes, for free resources vs 100$/month + resource cost for a startup, its worth it.
We should instead diversify by using multiple vendors for payment gateway .
I think that is an acceptable risk for free resources vs 100$/month or more investment for a startup that may or may not succeed.

4. Scale consideration: Lambda stays FREE until 267K orders/month?
1 mil requests/month
400k Gb secs/month(requests not orders.)

Write one paragraph: Should your startup switch to Lambda? Why or why not?
Yes, our startup should switch to Lambda for the payment processing system. The experimental results demonstrate that the cold start overhead (73.52ms) is negligible compared to our 3-second processing delay, making the performance impact virtually unnoticeable to customers. From a business perspective, the serverless architecture eliminates the operational burden of managing ECS worker scaling and SQS queue tuning, allowing the team to focus on core product development rather than infrastructure maintenance. Most importantly, Lambda's pay-as-you-go model, combined with a generous free tier, makes it significantly more cost-effective than maintaining 'always-on' ECS tasks, especially for a startup handling unpredictable traffic spikes or low-to-medium order volumes.

Contributions:

Hong

- Prepare a brief document outlining the key metrics to focus on, and create tables for teammates to complete so results can be compared across each phase.
- Code + locust test
- Answer question:
    - What happens to your customers? For phase 1
    - Find the balance: What's the minimum workers needed to prevent queue buildup at 60 orders/second? For phase 5
    - Write one paragraph: Should your startup switch to Lambda? Why or why not?

Jingjing

- Code + locust test
- Deployed the AWS Lambda function and successfully sent asynchronous test orders.

- Performed system monitoring and analyzed execution behavior using CloudWatch log streams.
- Answering the question:
    - How often do cold starts occur?
    - What's the overhead?
    - Does this matter for 3-second payment processing?

Stella

- Executed the codebase and conducted load testing using Locust.
- Analyzed system performance and identified bottlenecks for Phase 2.
- Performed CloudWatch monitoring and analysis for Phase 5, comparing asynchronous vs. synchronous architectures, evaluating queue buildup, and proposing prevention strategies.

Nithish
code(self)+test(self)+
Answered questions following for aggregated test results ":
1.How often did cold starts occur? Every few minutes? Every request?
2.Is the cost advantage compelling?
3.Can you accept losing SQS guarantees?
4.Scale consideration: